

ERA Tutorium 8

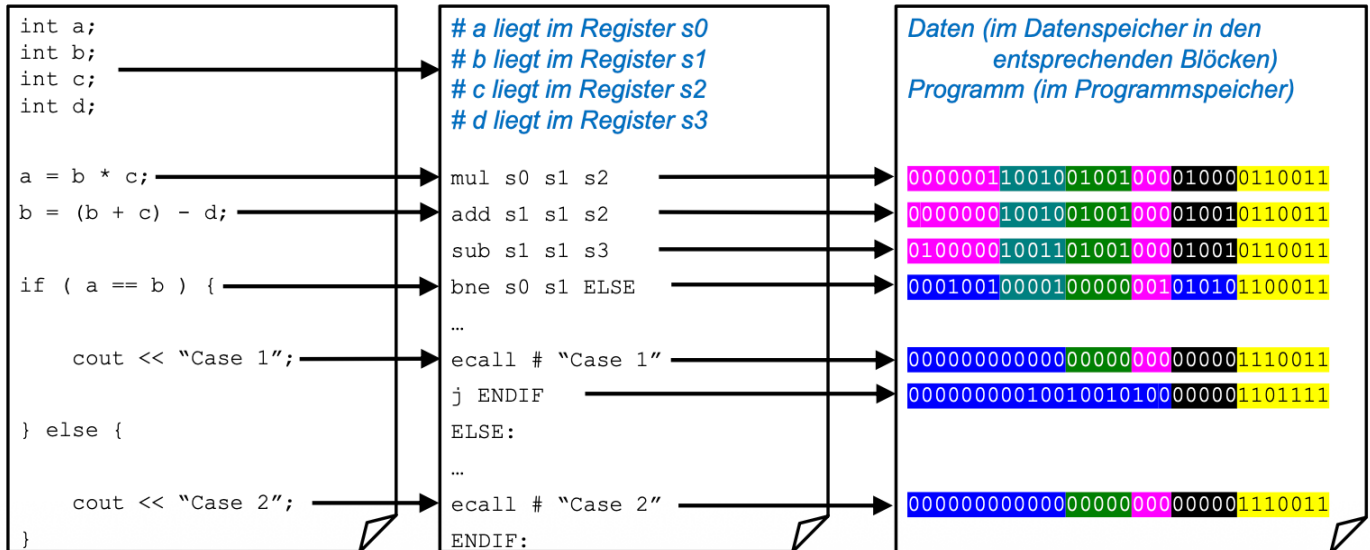
Leopold Cario	<u>Termin:</u>	<u>Raum:</u>	<u>Zulip:</u>
	· Mittwoch 10:00	03.13.010	ERA Tutorium - Mi - 1000-1
	· Donnerstag 12:00	00.08.059	ERA Tutorium - Do - 1200-1

Maschinensprache:

■ Höhere Programmiersprache

■ Assembler

■ Maschinensprache



Name	Nummer	Beschreibung
zero	x0	Konstante 0 (kann nicht überschrieben werden)
ra	x1	Rücksprungadresse
sp	x2	Stack-Zeiger
gp	x3	Zeiger auf globale Daten
tp	x4	Zeiger auf thread-lokale Daten
t0-2	x5 - 7	Temporäre Register*
s0/fp	x8	Stack-Frame-Zeiger (optional)**
s1	x9	Gesicherte Register**
a0-1	x10 - 11	Funktionsargumente/Rückgabewerte
a2-7	x12 - 17	Funktionsargumente
s2-11	x18 - 27	Gesicherte Register**
t3-6	x28 - 31	Temporäre Register*

(b) Registertabelle

$$add \quad s0, s1, s2 \quad = \quad \boxed{0000000} \boxed{10010} \boxed{01001} \boxed{000} \boxed{01000} \boxed{0110011} \quad = \quad 0x07248433$$

func7
rs2
rs1
func3
rd
op

31:25	24:20	19:15	14:12	11:7	6:0	
func7	rs2	rs1	func3	rd	op	R-Type
imm _{11:0}		rs1	func3	rd	op	I-Type
imm _{12:10:5}	rs2	rs1	func3	imm _{4:0}	op	S-Type
imm _{31:12}				rd	op	B-Type
imm _{20:10:11,19:12}				rd	op	U-Type
fs3	func2	fs2	fs1	func3	fd	J-Type
5 bits	2 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- imm: signed immediate in imm_{11:0}
- uimm: 5-bit unsigned immediate in imm_{4:0}
- upimm: 20 upper bits of a 32-bit immediate, in imm_{31:12}
- Address: memory address: rs1 + SignExt(imm_{11:0})
- [Address]: data at memory location Address
- BTA: branch target address: PC + SignExt((imm_{12:11}, 1'b0))
- JTA: jump target address: PC + SignExt((imm_{20:11}, 1'b0))
- label: text indicating instruction address
- SignExt: value sign-extended to 32 bits
- ZeroExt: value zero-extended to 32 bits
- csr: control and status register

op	func3	func7	Type	Instruction	Description	Operation
0000011 (3)	000	-	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})
0000011 (3)	001	-	I	lh rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})
0000011 (3)	010	-	I	lw rd, imm(rs1)	load word	rd = [Address] _{31:0}
0000011 (3)	100	-	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address] _{7:0})
0000011 (3)	101	-	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address] _{15:0})
0010011 (19)	000	-	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	-	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	-	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	-	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srli rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srai rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	-	I	ori rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	-	I	andi rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	-	-	U	auipc rd, upimm	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	-	S	sb rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011 (35)	001	-	S	sh rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011 (35)	010	-	S	sw rs2, imm(rs1)	store word	[Address] _{31:0} = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 - rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srli rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	-	-	U	lui rd, upimm	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	-	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	-	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	-	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	-	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	-	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	-	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	-	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	-	-	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

* Encoded in instr_{31:25}, the upper seven bits of the immediate field

Pseudoinstruction	RISC-V Instructions	Description	Operation
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called "register copy")	rd = rs1
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a label and a global variable, respectively.

Abbildung 4: RISC-V Befehlsreferenz

Lehrstuhl für
Rechnerarchitektur & Parallele Systeme
Prof. Dr. Martin Schulz
Dominic Prinz
Jakob Schöffeler

Lehrstuhl für
Design Automation
Prof. Dr.-Ing. Robert Wille
Stefan Engels
Benjamin Hien

Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

Übungsblatt 8: RISC-V Prozessor

08.12.2025 – 12.12.2025

1 Maschinensprache

a) Übersetze das folgende RISC-V Assembly Programm in RISC-V Maschinensprache

```
add s7, s8, s9
sll t1, t2, t3
srai s3, s1, 14
sw s9, 16(t4)
```

sll t1, t2, t3:

funct7 rs2 rs1 funct3 rd op
 0000000 1100 00111 001 00110 0110011
 = 0x01c39333

31:25		24:20		19:15		14:12		11:7		6:0			
funct7		rs2		rs1		funct3		rd		op		R-Type	
imm _{11:0}				rs1		funct3		rd		op		I-Type	
imm _{11:5}		rs2		rs1		funct3		imm _{4:0}		op		S-Type	
imm _{12,10:5}		rs2		rs1		funct3		imm _{4:1,11}		op		B-Type	
imm _{31:12}								rd		op		U-Type	
imm _{20,10:1,11,19:12}								rd		op		J-Type	
fs3		funct2		fs2		fs1		funct3		fd		op	R4-Type
5 bits		2 bits		5 bits		5 bits		3 bits		5 bits		7 bits	

srai s3, s1, 14.

funct7 imm rs1 funct3 rd op
 0100000 01110 01001 101 10011 0010011
 = 0x40e4d993

sw s9, 16(t4):

imm rs2 rs1 funct3 imm op
 0000000 11001 11101 010 10000 0100011
 = 0x019ea823

$$add \quad s0, s1, s2 \quad = \quad \boxed{0000000} \boxed{10010} \boxed{01001} \boxed{000} \boxed{01000} \boxed{0110011} = 0x07248433$$

func7
rs2
rs1
func3
rd
op

31:25 24:20 19:15 14:12 11:7 6:0						R-Type I-Type S-Type B-Type U-Type J-Type R4-Type
func7		rs2	rs1	func3	rd	
imm _{11:0}			rs1	func3	rd	
imm _{12:10:5}		rs2	rs1	func3	imm _{4:0}	
imm _{31:12}					rd	
imm _{20:10:11,19:12}					rd	
fs3	func2	fs2	fs1	func3	fd	
5 bits		2 bits	5 bits	5 bits	3 bits	5 bits
		5 bits	5 bits	3 bits	5 bits	7 bits

- imm: signed immediate in imm_{11:0}
- uimm: 5-bit unsigned immediate in imm_{4:0}
- upimm: 20 upper bits of a 32-bit immediate, in imm_{31:12}
- Address: memory address: rs1 + SignExt(imm_{11:0})
- [Address]: data at memory location Address
- BTA: branch target address: PC + SignExt((imm_{12:11}, 1'b0))
- JTA: jump target address: PC + SignExt((imm_{20:11}, 1'b0))
- label: text indicating instruction address
- SignExt: value sign-extended to 32 bits
- ZeroExt: value zero-extended to 32 bits
- csr: control and status register

op	func3	func7	Type	Instruction	Description	Operation
0000011 (3)	000	-	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})
0000011 (3)	001	-	I	lh rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})
0000011 (3)	010	-	I	lw rd, imm(rs1)	load word	rd = [Address] _{31:0}
0000011 (3)	100	-	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address] _{7:0})
0000011 (3)	101	-	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address] _{15:0})
0010011 (19)	000	-	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	-	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	-	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	-	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srli rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srai rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	-	I	ori rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	-	I	andi rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	-	-	U	auipc rd, upimm	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	-	S	sb rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011 (35)	001	-	S	sh rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011 (35)	010	-	S	sw rs2, imm(rs1)	store word	[Address] _{31:0} = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 - rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srli rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	-	-	U	lui rd, upimm	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	-	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	-	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	-	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	-	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	-	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	-	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	-	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	-	-	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

* Encoded in instr_{31:25}, the upper seven bits of the immediate field

Pseudoinstruction	RISC-V Instructions	Description	Operation
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called "register copy")	rd = rs1
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a label and a global variable, respectively.

Abbildung 4: RISC-V Befehlsreferenz

b) Konvertiere den folgenden RISC-V-Maschinencode zurück in RISC-V Assembly:

00: 0x01200513
 04: 0x00300593
 08: 0x00000393
 12: 0x00058e33
 16: 0x01c54863
 20: 0x00138393
 24: 0x00be0e33
 28: 0xff5ff06f
 32: 0x00038533

31:25		24:20		19:15		14:12		11:7		6:0		
funct7		rs2		rs1		funct3		rd		op		R-Type
imm _{11:0}				rs1		funct3		rd		op		I-Type
imm _{11:5}		rs2		rs1		funct3		imm _{4:0}		op		S-Type
imm _{12,10:5}		rs2		rs1		funct3		imm _{4:1,11}		op		B-Type
imm _{31:12}								rd		op		U-Type
imm _{20,10:1,11,19:12}								rd		op		J-Type
fs3	funct2	fs2	fs1	funct3		fd		op		R4-Type		
5 bits	2 bits	5 bits	5 bits	3 bits		5 bits		7 bits				

c) Erkläre was das Programm aus Teilaufgabe b) berechnet.

00: 0x01200513 = 0000 0001 0010 | 0000 0000 | 0101 0001 0011

I
 addi a0, zero, 18

16: 0x01c54863 = 0000 0001 1100 | 0101 0100 | 1000 0110 0011
 (10) (00000) (1000) (0)

B

blt a0, t3, 32
 00000000 10000 = 16

28: 0xff5ff06f = 1111 1111 0101 1111 1111 0000 0110 1111

_start:

```
addi a0, zero, 18
addi a1, zero, 3
addi t2, zero, 0
add t3, a1, zero
```

loop_start:

```
blt a0, t3, loop_end
addi t2, t2, 1
add t3, t3, a1
jal zero, loop_start
```

loop_end:

```
add a0, t2, zero
```

Befehl	Befehlstyp	Branch	ResultSrc	MemWrite	ALUControl	ALUSrc	ImmSrc	RegWrite
OR	R-Typ	0	00	0	OR	0	xx	1
BNE	B-Typ	1	xx	0	SUB	0	B-Typ	0
XORI	I-Typ	0	00	0	XOR	1	I-Typ	1
SW	S-Typ	0	xx	1	ADD	1	S-Typ	0

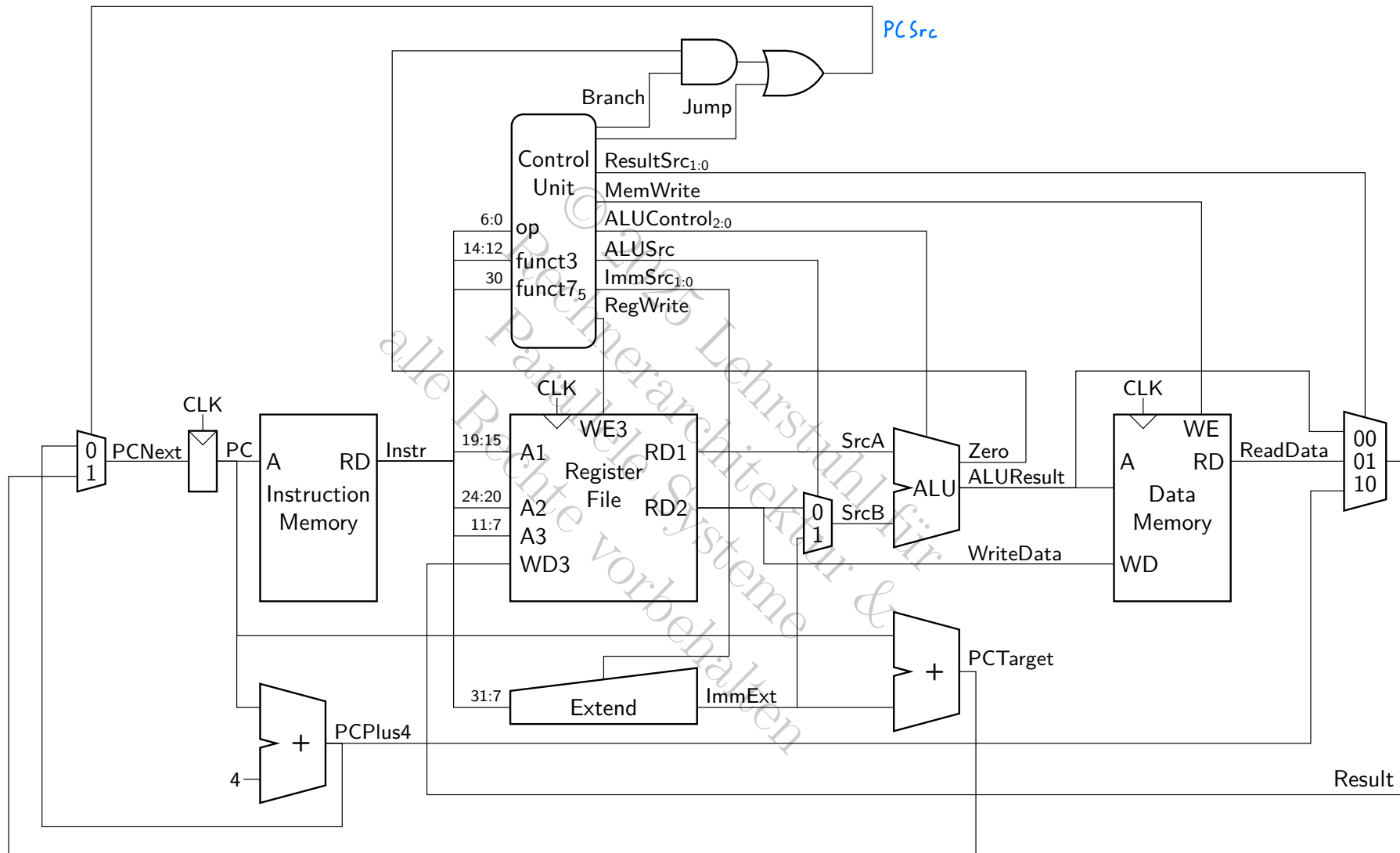


Abbildung 3: Schaltbild des Single Cycle RISC-V Prozessors

2 Fehlerhafte Kontrollsignale

Angenommen der Prozessor in Abbildung 3 hat fehlerhafte Kontrollsignale, deren Wert konstant auf einen bestimmten Wert gelegt ist. Die folgende Tabelle listet Kontrollsignale und deren konstanten Wert auf. Welche der Befehle, die in der Vorlesung vorgestellt wurden 1 funktionieren nicht mehr unter dieser fehlerhaften Belegung?

Kontrollsignal	Konstante Belegung	Fehlerhafte Befehle	Funktionale Befehle
ALUControl	000 (ADD)	BEQ, R-Typ außer add, I-Typ außer addi	lw, sw, jal, addi, add
PCSrc	0	BEQ, JAL	lw, sw, R-Typ, I-Typ ALU
ResultSrc	01	R-Typ, I-Typ ALU, JAL	lw, sw, beq

© 2025 Lehrstuhl für
Rechnerarchitektur &
Parallele Systeme
alle Rechte vorbehalten

¹lw, sw, R-Typ, I-Typ ALU, jal und beq

$$add \quad s0, s1, s2 \quad = \quad \boxed{0000000} \boxed{10010} \boxed{01001} \boxed{000} \boxed{01000} \boxed{0110011} = 0x07248433$$

func7
rs2
rs1
func3
rd
op

31:25 24:20 19:15 14:12 11:7 6:0							
func7	rs2	rs1	func3	rd	op		R-Type
imm _{11:0}		rs1	func3	rd	op		I-Type
imm _{12:10:5}	rs2	rs1	func3	imm _{4:0}	op		S-Type
imm _{31:12}				rd	op		B-Type
imm _{20:10:11,19:12}				rd	op		U-Type
fs3	func2	fs2	fs1	func3	fd	op	J-Type
5 bits	2 bits	5 bits	5 bits	3 bits	5 bits	7 bits	R4-Type

- imm: signed immediate in imm_{11:0}
- uimm: 5-bit unsigned immediate in imm_{4:0}
- upimm: 20 upper bits of a 32-bit immediate, in imm_{31:12}
- Address: memory address: rs1 + SignExt(imm_{11:0})
- [Address]: data at memory location Address
- BTA: branch target address: PC + SignExt((imm_{12:11}, 1'b0))
- JTA: jump target address: PC + SignExt((imm_{20:11}, 1'b0))
- label: text indicating instruction address
- SignExt: value sign-extended to 32 bits
- ZeroExt: value zero-extended to 32 bits
- csr: control and status register

op	func3	func7	Type	Instruction	Description	Operation
0000011 (3)	000	-	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})
0000111 (3)	001	-	I	lh rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})
0000111 (3)	010	-	I	lw rd, imm(rs1)	load word	rd = [Address] _{31:0}
0000111 (3)	100	-	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address] _{7:0})
0000111 (3)	101	-	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address] _{15:0})
0010011 (19)	000	-	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	-	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	-	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	-	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srli rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srai rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	-	I	ori rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	-	I	andi rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	-	-	U	auipc rd, upimm	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	-	S	sb rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011 (35)	001	-	S	sh rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011 (35)	010	-	S	sw rs2, imm(rs1)	store word	[Address] _{31:0} = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 - rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srli rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	-	-	U	lui rd, upimm	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	-	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	-	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	-	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	-	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	-	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	-	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	-	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	-	-	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

* Encoded in instr_{31:25}, the upper seven bits of the immediate field

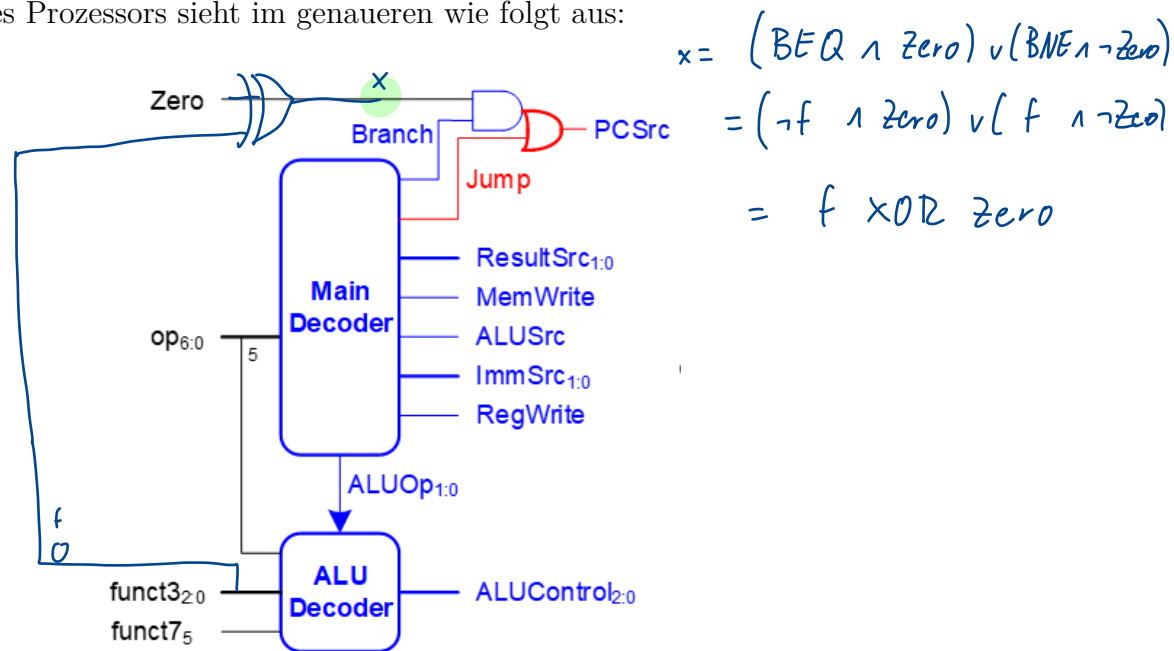
Pseudoinstruction	RISC-V Instructions	Description	Operation
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called "register copy")	rd = rs1
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a label and a global variable, respectively.

Abbildung 4: RISC-V Befehlsreferenz

3 Erweiterung: BNE

Das Steuerwerk des Prozessors sieht im genaueren wie folgt aus:



Insbesondere ist die Logik des PCSrc Signals genauer abgebildet: PCSrc wird auf 1 gesetzt wenn Jump auf 1 (JAL) ist oder bei einem Branch das Zero Signal 1 liefert (BEQ).

Erweitere das Steuerwerk und gegebenenfalls das Prozessorschaltbild so, dass auch ein BNE (Branch Not Equal) Befehl durchgeführt werden kann. Der Maschinencode des BNE ist in Abbildung 4 abgebildet.

4 Kontrollsignalbelegung

Gebe die Belegung der Kontrollsignale für folgende Befehle an. Für ALUControl und ImmSrc reicht der entsprechende Name der Funktion bzw. des Typs. Gebe Signale bei denen die Belegung keinen Einfluss auf die Funktionalität hat explizit mit *Don't Care* ('x') an.

Befehl	Befehlstyp	Branch	ResultSrc	MemWrite	ALUControl	ALUSrc	ImmSrc	RegWrite
OR								
BNE								
XORI								
SW								

5 Prozessorerweiterung (Hausaufgabe)

Bearbeitung und Abgabe der Hausaufgabe 8 auf <https://artemis.tum.de/courses/516> bis Sonntag, den 14.12.2025, 23:59 Uhr.